

Plantillas Angular

Desarrollo Web en Entorno Cliente

Daniel Suárez Martín

Curso 25/26

Índice

Contenido

Índice	2
Introducción	3
1. Modificación del fichero 'Generaciones.ts'	3
2. Modificación del fichero 'Generaciones.html'	4
3. Implementación de 'Bulma' y modificación del estilo	4
CONCLUSIÓN	6

Introducción

Esta práctica de la asignatura consistirá en poner en práctica los conocimientos adquiridos sobre el uso de Angular CLI y la creación de componentes dentro de una aplicación Angular. Partiendo del proyecto previamente desarrollado, *PokemonManager*, se trabajará con los componentes ya creados para ampliar su funcionalidad y mejorar la presentación de la información en la página principal.

El objetivo principal será integrar el componente de generaciones en la vista de inicio de la aplicación, creando y gestionando un array con las distintas generaciones de Pokémon y mostrándolas dinámicamente mediante el uso de directivas estructurales en la plantilla. Además, se incorporará el framework de estilos *Bulma*, con el fin de mejorar la apariencia visual de la lista y reforzar la comprensión del uso de dependencias externas en proyectos Angular.

1. Modificación del fichero 'Generaciones.ts'

Como hemos mencionado anteriormente, en esta práctica ampliaremos la funcionalidad del componente 'Generaciones', para que muestre una lista de todas las generaciones Pokémon mediante el uso de directivas. Para ello, el primer paso será acudir al archivo 'generaciones.ts' e importar los módulos 'CommonModule' y 'OnInit'.

```
import { CommonModule } from '@angular/common';
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-generaciones',
  imports: [CommonModule],
  templateUrl: './generaciones.html',
  styleUrls: ['./generaciones.css'],
})
```

El primero será el que nos permita el uso de directivas más adelante, mientras que el segundo es una interfaz que nos permitirá ejecutar el código que desarrollemos dentro de este archivo en el fichero HTML del mismo componente.

A continuación, dentro de nuestra clase Generaciones, agregamos la interfaz 'OnInit' añadiendo 'implements OnInit', un constructor vacío y el método 'ngOnInit' (el cual, para esta práctica, no tocaremos en lo absoluto).

```
export class Generaciones implements OnInit {

  //Agregamos el constructor vacío
  constructor() { }

  //Agregamos el método ngOnInit, también sin desarrollar
  ngOnInit(): void {
  }
```

El último paso dentro de este fichero será la creación de nuestro array mediante una variable, a la cual denominaré como 'generaciones', la cual usaremos para mostrar la información de la misma en nuestra aplicación.

```
generaciones: string[] =  
  'Generación I',  
  'Generación II',  
  'Generación III',  
  'Generación IV',  
  'Generación V',  
  'Generación VI',  
  'Generación VII',  
];
```

2. Modificación del fichero 'Generaciones.html'

Una vez tenemos editado nuestro archivo *TypeScript*, el siguiente paso será la modificación del fichero HTML. Para mostrar la información de nuestro array como una lista, nos valdremos de la directiva estructural '*ngFor', la cual nos permitirá recorrer dicho array y mostrar cada elemento mediante la interpolación de datos realizada con el elemento de llaves dobles '{{}}':

```
<li *ngFor="let generacion of generaciones" class="has-text-black mb-2">{{ generacion }}</li>
```

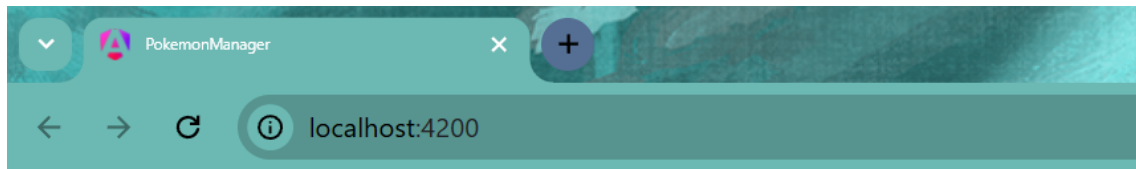
Como se aprecia en el código, lo que estamos haciendo es recorrer el array y sacar cada elemento de forma individual dentro de una etiqueta de listado 'li'.

3. Implementación de 'Bulma' y modificación del estilo

Una vez la lógica de nuestro programa funciona para darnos el objetivo deseado, es hora de aplicar los estilos correspondientes. Para ello, nos valdremos del *framework* Bulma. Para poder utilizarlo, el primer paso será su instalación mediante el comando en Terminal 'npm install bulma'. Una vez se han terminado de instalar correctamente, el siguiente paso será modificar el fichero 'angular.json' para incluir este *framework* como fuente de estilos. Para ello, buscaremos el elemento 'styles' y le añadiremos la ruta de la fuente de estilos de Bulma:

```
"styles": [  
  "src/styles.css",  
  "node_modules/bulma/css/bulma.min.css"
```

Una vez añadida, podremos pasar a modificar la apariencia de nuestra aplicación utilizando clases predefinidas de Bulma, como puede ser el reparto del espacio de un contenedor, el color de fuente, su tamaño o el color de fondo. Una vez hechas todas las modificaciones, el resultado final obtenido es el siguiente:



Mi aplicación de Angular
Bienvenido a mi aplicación de gestión de Pokémons.

Generaciones de Pokémon:

Listado de todas las Generaciones:

- Generación I
- Generación II
- Generación III
- Generación IV
- Generación V
- Generación VI
- Generación VII

Componente **Pokémons** creado correctamente.

CONCLUSIÓN

El desarrollo de esta práctica me ha resultado útil para afianzar los conocimientos adquiridos sobre Angular y comprender mejor la estructura y funcionamiento de sus componentes. Aunque el ejercicio planteado no era especialmente complejo a nivel lógico, sí ha servido para reforzar conceptos fundamentales como la creación y modificación de componentes, el uso de directivas estructurales y la vinculación dinámica de datos entre el archivo 'TypeScript' y su correspondiente plantilla HTML.

Además, la utilización de la directiva '`*ngFor`' para recorrer un array y mostrar su contenido mediante interpolación me ha permitido entender de una forma más práctica cómo Angular gestiona el renderizado dinámico de información. Del mismo modo, el hecho de trabajar con la interfaz '`OnInit`', aunque en esta práctica no se haya explotado su funcionalidad, ayuda a familiarizarse con el ciclo de vida de los componentes, algo que será clave en desarrollos más complejos.

Por otra parte, la instalación e integración del *framework* 'Bulma' ha supuesto un primer acercamiento a la incorporación de dependencias externas en un proyecto Angular. Este proceso no solo mejora el apartado visual de la aplicación, sino que también aporta una visión más profesional del desarrollo *frontend*, al trabajar con *frameworks* de estilos que optimizan tiempo y organización.

En líneas generales, considero que la práctica ha sido positiva, ya que consolida la base sobre la que se construirán funcionalidades más avanzadas en el futuro. Aunque el resultado visual sea sencillo, el aprendizaje técnico detrás de cada paso realizado aporta una mayor seguridad y comprensión del entorno de trabajo con Angular.